

Chapter 33: LangGraph Deployment Architectures, CI/CD, GitOps, Kubernetes, Containerization, Environment Promotion, and Production Release Management

Part III: LangGraph Deep Dive

Book: Agentic AI & RAG Systems Handbook

Version: 2.0 Expanded Edition

Chapter ID: 33

File Name: chapter-33-langgraph-deployment-and-devops.md

Chapter Overview

Building a LangGraph workflow is only the beginning.

The real challenge is:

```
1  How do we safely deploy,  
2  
3  operate,  
4  
5  upgrade,  
6  
7  rollback,  
8  
9  and scale  
10  
11 enterprise agent systems?
```

Many AI projects succeed in development but fail during production deployment.

Reasons include:

- No CI/CD strategy
- Poor environment management
- Manual deployments
- Lack of rollback support
- Missing governance controls
- Inconsistent prompts across environments

- Uncontrolled model updates

Enterprise LangGraph systems require mature software delivery practices integrated with AI-specific governance.

This chapter focuses on production deployment architectures and operational excellence.

Learning Objectives

After completing this chapter you should be able to:

- Design enterprise deployment architectures
 - Implement CI/CD pipelines for LangGraph
 - Apply GitOps principles
 - Containerize agent systems
 - Deploy LangGraph on Kubernetes
 - Design environment promotion models
 - Implement release management controls
 - Create rollback strategies
 - Explain deployment architectures during Staff and Principal interviews
-

Prerequisites

Required:

- Chapter 28: Observability
 - Chapter 29: Error Handling & Resilience
 - Chapter 30: Performance Engineering
 - Chapter 31: Security
 - Chapter 32: Testing & Evaluation
-

1. The Deployment Challenge

Prototype workflow:

1	Developer Laptop
2	↓
3	Works

Production workflow:

1	Source Code
2	↓
3	Build
4	↓
5	Test
6	↓
7	Security Scan
8	↓
9	Evaluation
10	↓
11	Deploy
12	↓
13	Monitor
14	↓
15	Rollback If Needed

Enterprise deployments are significantly more complex.

2. Deployment Principles

Principle 1

Automate Everything

Principle 2

Version Everything

Principle 3

Promote Through Environments

Principle 4

Deploy Safely

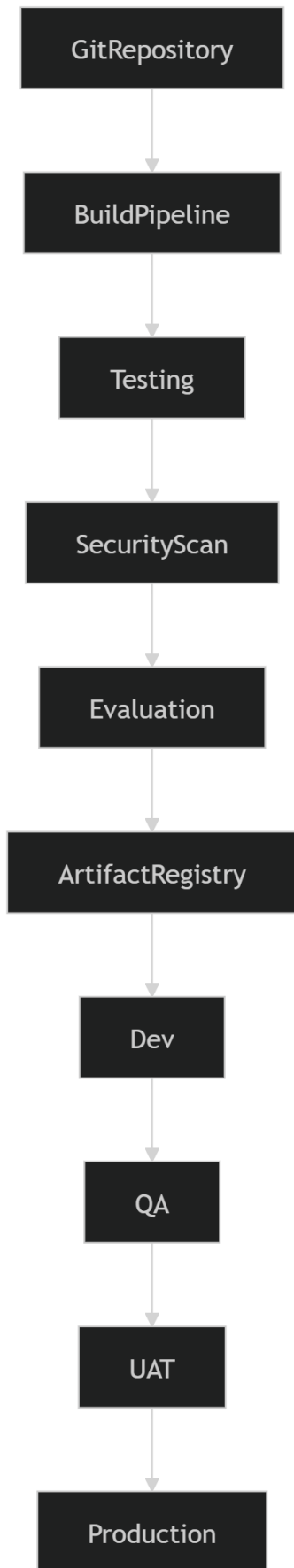
Principle 5

Observe Continuously

Principle 6

Rollback Quickly

3. Enterprise Deployment Architecture



4. Delivery Pipeline Overview

The complete deployment lifecycle:

```
1  Code
2
3  ↓
4
5  Build
6
7  ↓
8
9  Test
10
11 ↓
12
13 Evaluate
14
15 ↓
16
17 Package
18
19 ↓
20
21 Deploy
22
23 ↓
24
25 Observe
26
27 ↓
28
29 Improve
```

5. Source Control Strategy

Everything should be managed in Git.

Version Control

Track:

```
1  Code
2
3  Prompts
4
5  Configurations
6
7  Policies
8
9  Schemas
10
11 Infrastructure
```

Repository Structure

```
1  /
2  |— graphs/
3  |— prompts/
4  |— policies/
5  |— evaluations/
6  |— infrastructure/
7  |— tests/
8  └— deployment/
```

6. Infrastructure as Code

Enterprise deployments require IaC.

Common tools:

```
1  Terraform
2
3  Bicep
4
5  CloudFormation
6
```


7 Pulumi

Benefits:

- 1 Repeatability
- 2
- 3 Auditability
- 4
- 5 Consistency

7. Containerization

Containerization is the preferred deployment model.

Architecture:

- 1 LangGraph
- 2
- 3 +
- 4
- 5 Dependencies
- 6
- 7 +
- 8
- 9 Configuration
- 10
- 11 =
- 12
- 13 Container Image

Benefits

- 1 Portability
- 2
- 3 Consistency
- 4
- 5 Scalability

8. Docker Architecture

Example:

```
1 FROM python:3.12
2
3 COPY app /app
4
5 RUN pip install -r requirements.txt
6
7 CMD ["python","main.py"]
```

Container becomes deployment unit.

9. Kubernetes Fundamentals

Most enterprise LangGraph systems run on:

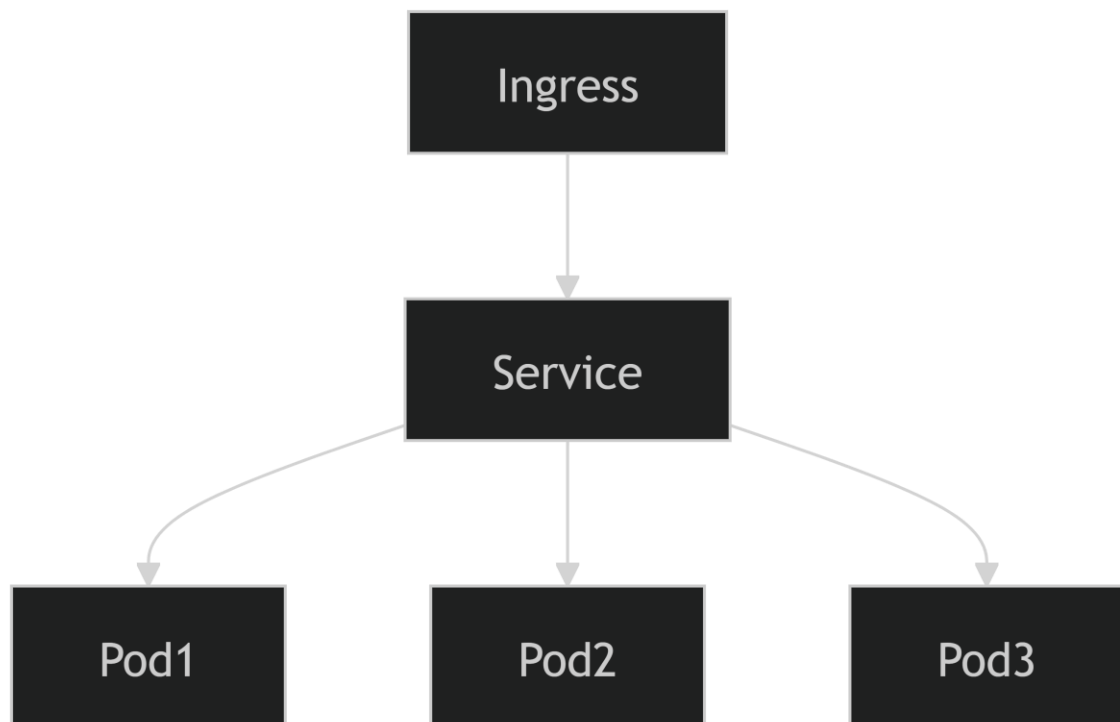
```
1 Kubernetes
```

or managed Kubernetes platforms.

Examples:

```
1 AKS
2
3 EKS
4
5 GKE
6
7 OpenShift
```

10. Kubernetes Deployment Model

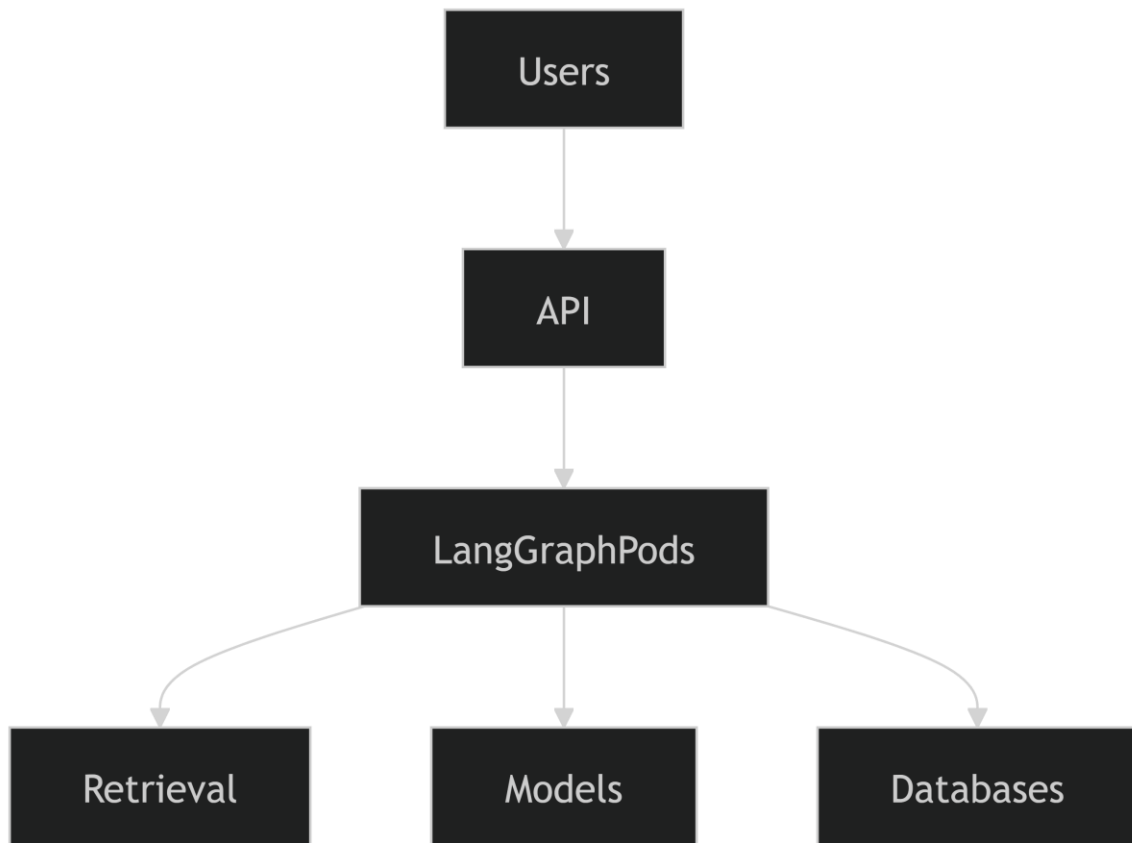


Benefits:

- | | |
|---|-------------------|
| 1 | High Availability |
| 2 | |
| 3 | Scaling |
| 4 | |
| 5 | Self Healing |

11. LangGraph on Kubernetes

Typical architecture:



12. Stateless vs Stateful Components

Stateless

Examples:

- | | |
|---|-----------------|
| 1 | Graph Workers |
| 2 | |
| 3 | API Services |
| 4 | |
| 5 | Execution Nodes |

Easy to scale.

Stateful

Examples:

- | | |
|---|---------------|
| 1 | Memory Stores |
|---|---------------|

- | | |
|---|------------------|
| 2 | |
| 3 | Checkpoints |
| 4 | |
| 5 | Vector Databases |

Require special treatment.

13. Environment Strategy

Never deploy directly to production.

Recommended environments:

- | | |
|---|------|
| 1 | DEV |
| 2 | |
| 3 | QA |
| 4 | |
| 5 | UAT |
| 6 | |
| 7 | PROD |

Flow

- | | |
|----|------|
| 1 | DEV |
| 2 | |
| 3 | ↓ |
| 4 | |
| 5 | QA |
| 6 | |
| 7 | ↓ |
| 8 | |
| 9 | UAT |
| 10 | |
| 11 | ↓ |
| 12 | |
| 13 | PROD |

14. Environment Promotion

Promotion should be controlled.

Example:

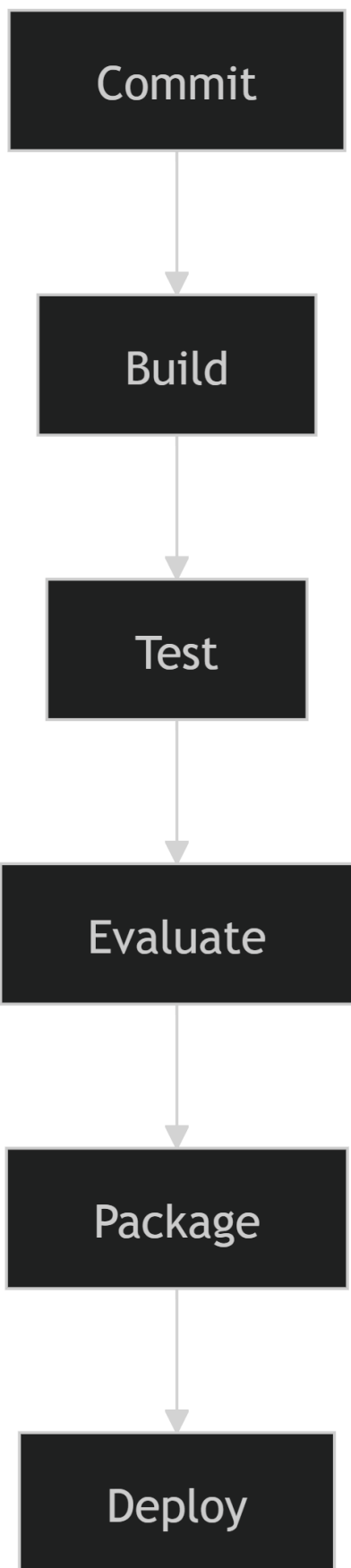
1	Tests Pass
2	
3	↓
4	
5	Security Scan Passes
6	
7	↓
8	
9	Evaluation Passes
10	
11	↓
12	
13	Promote

Never:

1	Deploy Directly
---	-----------------

without validation.

15. CI/CD Architecture



CI Responsibilities

- | | |
|---|----------|
| 1 | Build |
| 2 | |
| 3 | Test |
| 4 | |
| 5 | Scan |
| 6 | |
| 7 | Validate |

CD Responsibilities

- | | |
|---|---------|
| 1 | Deploy |
| 2 | |
| 3 | Promote |
| 4 | |
| 5 | Monitor |

16. AI-Specific CI/CD

Traditional pipelines are not enough.

Add:

Prompt Validation

Evaluation Testing

Hallucination Testing

Grounding Testing

Security Testing

Example:

- | | |
|---|-----------------------|
| 1 | Code Tests Pass |
| 2 | |
| 3 | But |
| 4 | |
| 5 | Grounding Score Drops |

Deployment blocked.

17. Evaluation Gates

Recommended production gate.

Example:

```
1  grounding_score:
2    minimum: 90%
3
4  hallucination_rate:
5    maximum: 5%
```

Deployment proceeds only if quality requirements are satisfied.

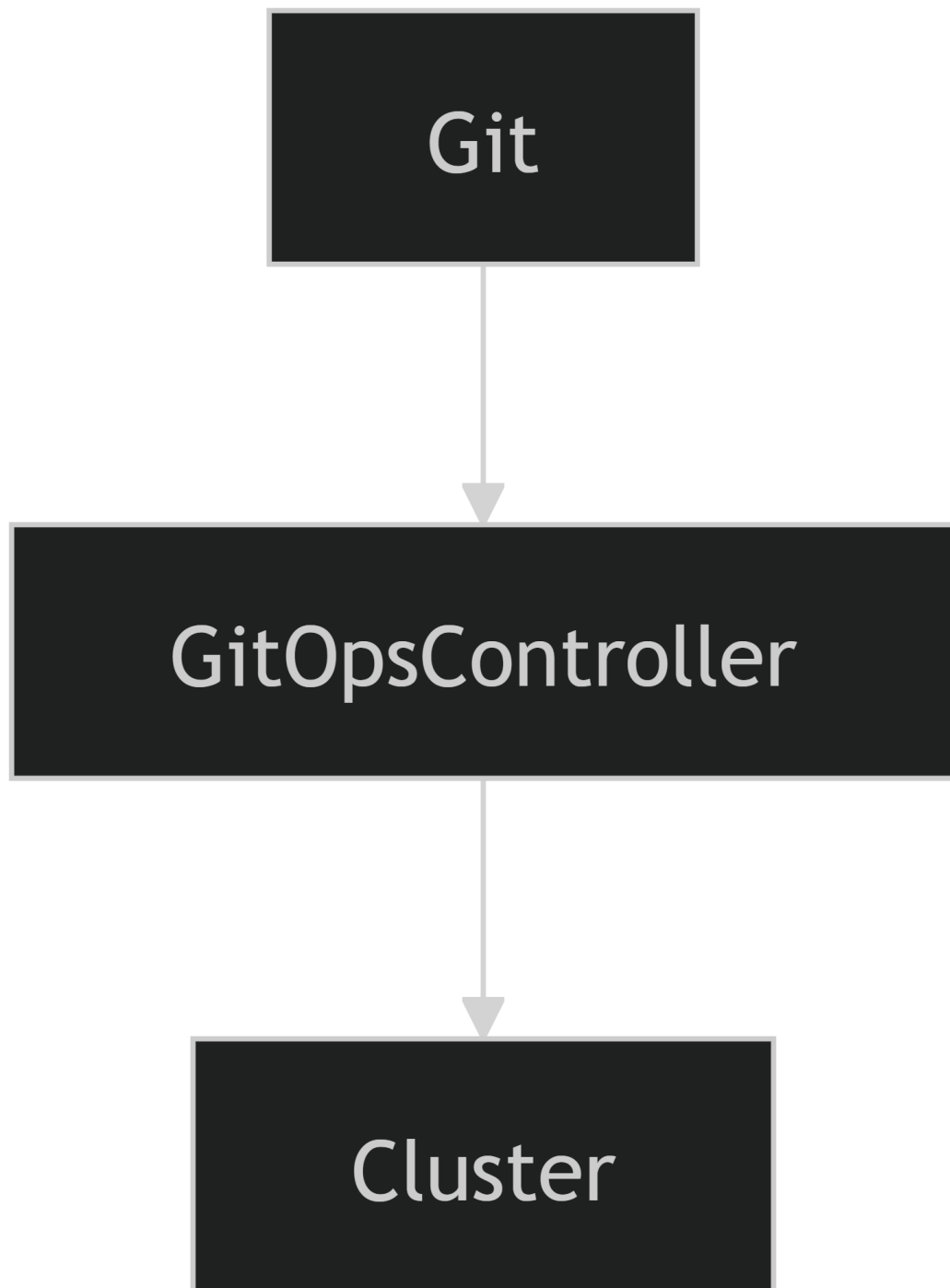
18. GitOps Architecture

GitOps is becoming the preferred enterprise approach.

Principle:

```
1  Git
2
3  =
4
5  Source Of Truth
```

Architecture:



Popular Tools:

- | | |
|---|--------|
| 1 | ArgoCD |
| 2 | |

19. GitOps Benefits

Advantages:

- | | |
|---|--------------|
| 1 | Auditability |
| 2 | |
| 3 | Rollback |
| 4 | |
| 5 | Consistency |
| 6 | |
| 7 | Governance |

Every deployment originates from Git.

20. Release Strategies

Multiple deployment approaches exist.

Big Bang

Deploy everything.

Highest risk.

Rolling Deployment

Gradual replacement.

Most common.

Blue-Green Deployment

Two environments.

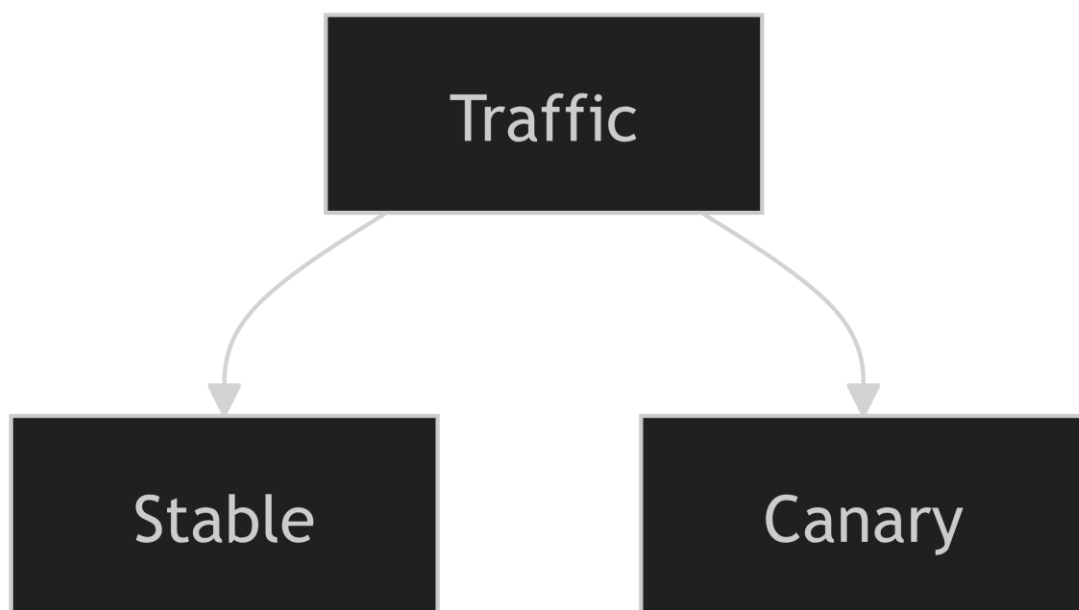
Canary Deployment

Small subset first.

Preferred for AI systems.

21. Canary Deployment

Architecture:



Example:

- | | |
|---|------------|
| 1 | 95% Stable |
| 2 | |
| 3 | 5% Canary |

Monitor performance before expanding.

22. Blue-Green Deployment

Maintain:

- | | |
|---|------------------|
| 1 | Blue Environment |
|---|------------------|

- | |
|---------------------|
| 2 |
| 3 Green Environment |

Traffic shifts when validation succeeds.

Benefits:

- | |
|-----------------|
| 1 Fast Rollback |
|-----------------|

23. Prompt Versioning

Prompts are deployment artifacts.

Treat them like code.

Track:

- | |
|----------------------|
| 1 Prompt Version |
| 2 |
| 3 Author |
| 4 |
| 5 Changes |
| 6 |
| 7 Evaluation Results |

24. Model Version Management

Models evolve frequently.

Examples:

- | |
|-----------------|
| 1 GPT Version A |
| 2 |
| 3 ↓ |
| 4 |
| 5 GPT Version B |

Changes require evaluation.

Never assume compatibility.

25. Configuration Management

Separate:

```
1 Code
2
3 Config
4
5 Secrets
```

Example:

```
1 model: gpt-5
2
3 temperature: 0.2
```

Configuration should not require code changes.

26. Secret Management

Never store:

```
1 API Keys
2
3 Passwords
4
5 Tokens
```

inside source code.

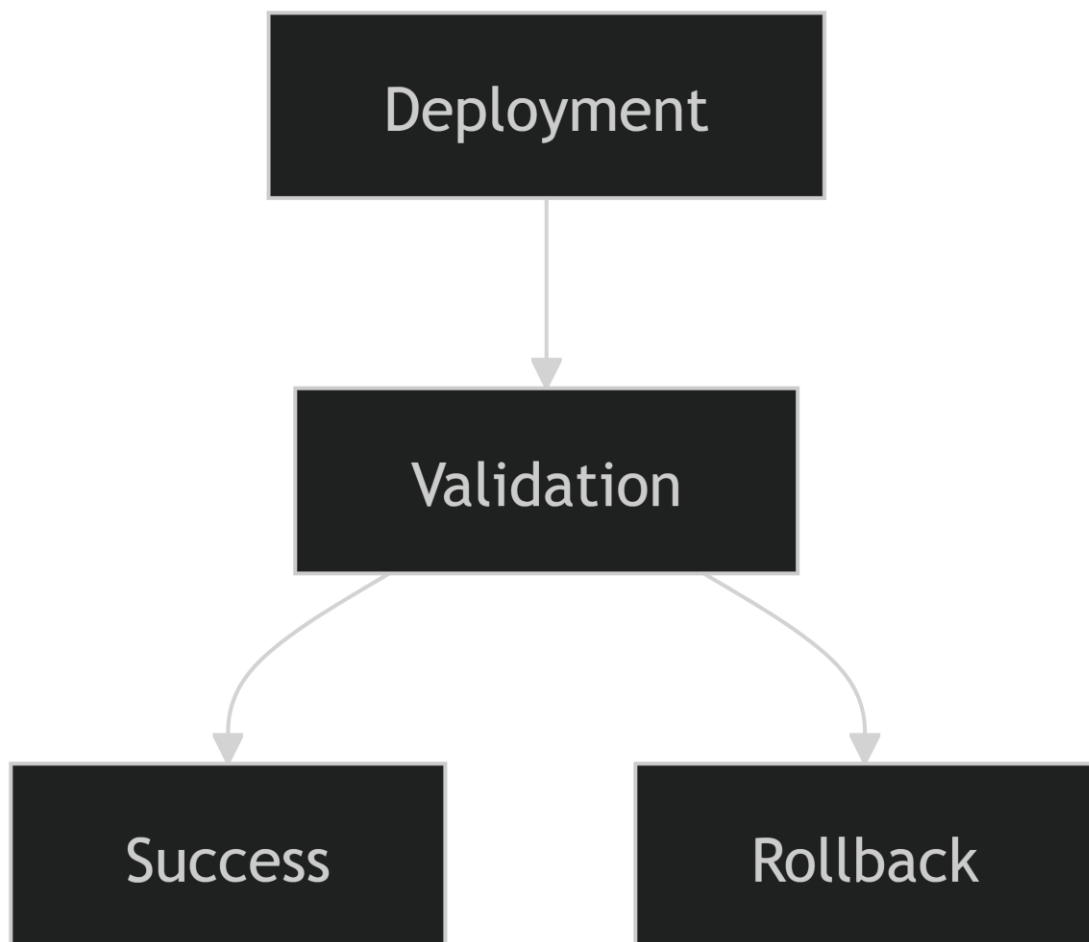
Use:

```
1 Azure Key Vault
2
3 AWS Secrets Manager
4
5 Vault
```

27. Rollback Strategy

Every deployment requires rollback.

Architecture:



Rollback targets:

- | | |
|---|----------|
| 1 | Code |
| 2 | |
| 3 | Prompts |
| 4 | |
| 5 | Models |
| 6 | |
| 7 | Policies |

28. Disaster Recovery

Plan for:

- | | |
|---|------------------|
| 1 | Region Failure |
| 2 | |
| 3 | Database Failure |
| 4 | |
| 5 | Model Failure |
| 6 | |
| 7 | Provider Outage |

Recovery objectives:

RTO

Recovery Time Objective

RPO

Recovery Point Objective

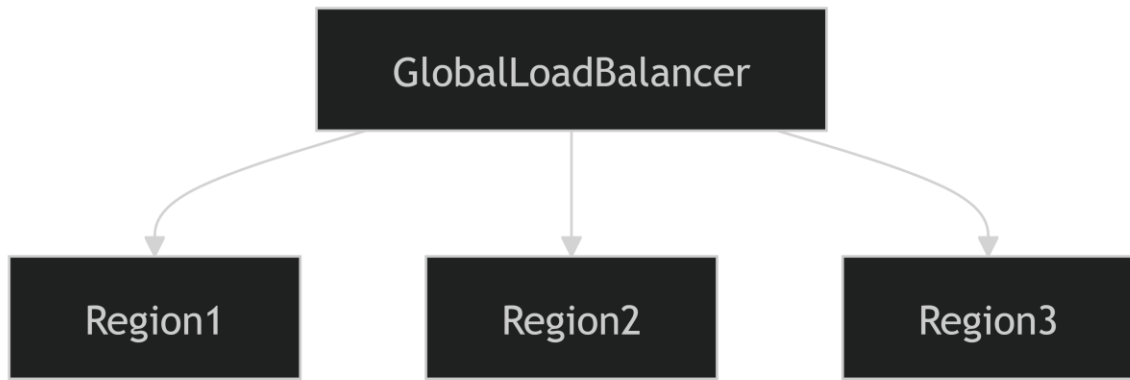
29. Multi-Region Deployment

Global systems often require:

- | | |
|---|--------|
| 1 | US |
| 2 | |
| 3 | Europe |
| 4 | |
| 5 | Asia |

deployments.

Architecture:



30. Observability Integration

Deployment is incomplete without monitoring.

Track:

Deployment Success

Rollbacks

Latency

Cost

Hallucinations

Failures

31. Operational Readiness

Production release checklist should include:

Security Approval

Architecture Approval

Evaluation Approval

Runbooks

Rollback Validation

32. SRE Integration

AI systems increasingly align with SRE practices.

Monitor:

- | | |
|---|--------------|
| 1 | Availability |
| 2 | |
| 3 | Latency |
| 4 | |
| 5 | Error Rate |
| 6 | |
| 7 | Saturation |

Plus AI metrics:

- | | |
|---|----------------|
| 1 | Grounding |
| 2 | |
| 3 | Hallucinations |
| 4 | |
| 5 | Quality |

33. Governance Controls

Deployment governance should include:

Change Approval

Audit Trails

Compliance Validation

Security Review

Critical systems require human approval.

34. Failure Modes

Failure 1

Unversioned Prompts

Failure 2

Manual Deployments

Failure 3

No Rollback Strategy

Failure 4

No Evaluation Gates

Failure 5

Environment Drift

35. Common Mistakes

Mistake 1

Treating prompts as configuration only.

Mistake 2

Skipping evaluation.

Mistake 3

No GitOps model.

Mistake 4

No deployment monitoring.

Mistake 5

No rollback testing.

36. Anti-Patterns

Deploy From Laptop

Never acceptable.

Production First

Skipping lower environments.

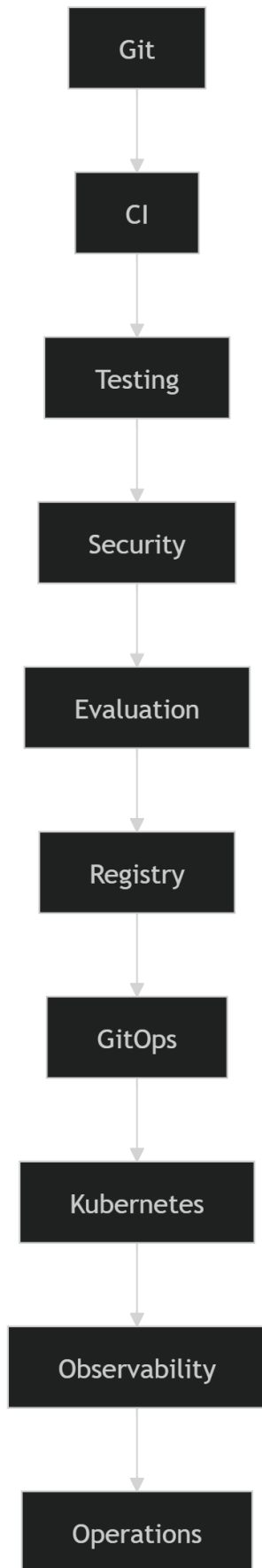
Shared Environments

Causes instability.

No Change Control

Creates governance risk.

37. Enterprise Reference Architecture



38. Architecture Decision Matrix

Deployment Capability	Importance
CI/CD	Critical
GitOps	Critical
Evaluation Gates	Critical
Canary Releases	High
Kubernetes	High
Rollback	Critical
Observability	Critical
Multi-Region Deployment	Context Dependent

39. Real-World Use Cases

Enterprise Copilots

Controlled prompt and model rollout.

Compliance Systems

Governed deployments.

Agentic Workforce Platforms

Canary-based upgrades.

IT Operations Systems

GitOps-driven releases.

Financial Systems

Strict promotion controls.

40. Interview Questions

Q1

Why is AI deployment different from traditional application deployment?

Q2

What additional CI/CD controls do AI systems need?

Q3

Why should prompts be versioned?

Q4

What is GitOps?

Q5

Why are evaluation gates important?

Q6

How would you deploy LangGraph on Kubernetes?

Q7

When should canary deployments be used?

Q8

Describe an enterprise deployment architecture for LangGraph.

41. Interview Answers

Q1

AI systems introduce prompts, models, retrieval systems, evaluations, and quality metrics that must be deployed and governed alongside code.

Q2

Prompt testing, grounding validation, hallucination checks, model evaluation, and security testing.

Q3

Because prompt changes can significantly alter system behavior and must be tracked, tested, and rolled back.

Q4

A deployment model where Git serves as the authoritative source of truth for system state.

Q5

They prevent degraded quality, hallucinations, or regressions from reaching production.

Q6

Using containerized LangGraph services deployed through Kubernetes with autoscaling, observability, and GitOps controls.

Q7

When introducing new prompts, workflows, models, or architectures with uncertain risk.

Q8

Git → CI/CD → Security Testing → Evaluation → Artifact Registry → GitOps → Kubernetes → Observability → Operations.

42. Architecture Review Checklist

Deployment Readiness Assessment

- CI/CD implemented
 - GitOps configured
 - Evaluation gates enforced
 - Security scans enabled
 - Prompts versioned
 - Models versioned
 - Rollback tested
 - Observability integrated
 - Runbooks documented
 - Disaster recovery validated
-

43. Practical Exercises

1. Design a GitOps deployment architecture for LangGraph.
 2. Create an evaluation gate strategy using LLM-as-a-Judge.
 3. Design a blue-green deployment model.
 4. Build a multi-environment promotion workflow.
 5. Create a rollback strategy for prompt and model upgrades.
-

44. Key Takeaways

- Deployment is a critical component of enterprise AI architecture.
 - LangGraph systems should be containerized and deployed through automated pipelines.
 - GitOps provides auditability, consistency, and operational control.
 - AI systems require evaluation gates in addition to traditional tests.
 - Prompts, models, and workflows must be versioned.
 - Canary deployments reduce rollout risk.
 - Rollback strategies should cover code, prompts, and models.
 - Kubernetes is the dominant enterprise deployment platform.
 - Observability must be integrated into deployment workflows.
 - Mature deployment practices are essential for production-grade agent platforms.
-

45. References

- [LangGraph Documentation](#)
 - [Kubernetes Documentation](#)
 - [ArgoCD Documentation](#)
 - [FluxCD Documentation](#)
 - [Google SRE Handbook](#)
 - [Azure Well-Architected Framework](#)
 - [AWS Well-Architected Framework](#)
 - [GitOps Working Group](#)
 - [CNCF Platform Engineering Guides](#)
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 34 – LangGraph Enterprise Governance, Responsible AI, Risk Management, Compliance Controls, Auditability, and AI Operating Models